Project Log Entry

FEC API Data Pipeline

Date: October 8, 2026
Project: FEC API Data Pipeline
Hours: 3 Hours 30 Minutes

Session Objective:
Build a data pipeline that ingests 2026 U.S. House and Senate candidate data from the OpenFEC API, stores the collected raw data in Amazon S3, transforms the data into an analytics-ready Parquet dataset, and queries the processed data with Amazon Athena.

AWS Services Used:
Amazon S3, AWS IAM, AWS Glue Data Catalog, Amazon Athena

Work Completed:
Connected to the OpenFEC API and configured the request to return active 2026 House and Senate candidate records. Built a Python ingestion script that authenticated with an API key, handled pagination across 43 pages, and collected all 4,292 records returned by the API. Validated the ingestion by comparing the API-reported record count with the number of records collected by the script.

Configured the AWS CLI and IAM credentials so the local Python environment could interact with AWS. Used Boto3 to upload the collected API results into an Amazon S3 raw-data layer as JSON.

Built a separate Python transformation script that read the raw JSON from S3, selected 18 fields for the curated dataset, converted the data to Parquet using pandas and PyArrow, and uploaded the processed file into a separate curated S3 location.

Created an Athena database and external table for the curated Parquet data, which registered the dataset metadata in the AWS Glue Data Catalog. Validated that all 4,292 records were preserved through the pipeline and ran initial analytical queries against fundraising-related flags and incumbent/challenger classifications. Saved screenshots throughout the process for project and GitHub documentation.

Problem / Decision:
The first local API request returned a 403 Forbidden error because the FEC API key was not available as an environment variable in the Terminal session. Identified the missing environment variable, configured the API key, and successfully authenticated without hard-coding the key into the Python script.

Also decided to use Python for the transformation instead of an AWS Glue ETL job. The dataset was small enough that a Glue Spark job would add unnecessary processing and cost. Keeping the

transformation in Python allowed the project to remain simple while still separating the raw JSON and curated Parquet layers.

What I Learned:
Gained hands-on experience consuming a REST API as a data source instead of manually uploading a file. Practiced API authentication, pagination, HTTP response handling, and validating ingestion completeness by comparing expected and collected record counts.

Also reinforced the purpose of separating raw and curated data in S3. The raw layer preserves the collected API data, while the curated layer contains a smaller, analysis-ready Parquet dataset. Practiced using Boto3 to interact with S3 programmatically and reinforced how Parquet supports efficient analytical workloads in Athena.

This project also helped connect API ingestion, transformation, storage, metadata, and SQL analysis into one complete data pipeline.

Evidence:
Python ingestion and transformation scripts, OpenFEC API response and record-count validation, S3 raw JSON and curated Parquet objects, Athena database and external table, Athena validation and analytical queries, and screenshots documenting major pipeline stages.

Pipeline Architecture:
OpenFEC API → Python → Amazon S3 Raw JSON → Python Transformation → Amazon S3 Curated Parquet → AWS Glue Data Catalog → Amazon Athena